



DISJOINT SET

PREPARED BY
SWAPNIL BISWAS



DISJOINT SET

□ A Disjoint Set **S** is a collection of sets $S_1, S_2, S_3, \dots, S_n$

- $\forall_{i \neq j} S_i \cap S_j = \emptyset$

□ Let $S_1 = \{a, b, c\}$, $S_2 = \{b, p, q\}$, $S_3 = \{e, f\}$, $S_4 = \{x\}$, $S_5 = \{c, q, y\}$

- Is $\{S_1, S_2\}$ disjoint? • **NO** • $S_1 \cap S_2 \neq \emptyset$
- Is $\{S_1, S_2, S_3\}$ disjoint? • **NO** • $S_1 \cap S_2 \neq \emptyset$
- Is $\{S_1, S_2, S_5\}$ disjoint? • **NO** • $S_1 \cap S_2 \neq \emptyset$ • $S_1 \cap S_5 \neq \emptyset$ • $S_2 \cap S_5 \neq \emptyset$
- Is $\{S_1, S_3\}$ disjoint? • **YES**
- Is $\{S_1, S_3, S_4\}$ disjoint? • **YES**
- Is $\{S_1\}$ disjoint? • **YES**

□ Each set has a representative which is the member of that set

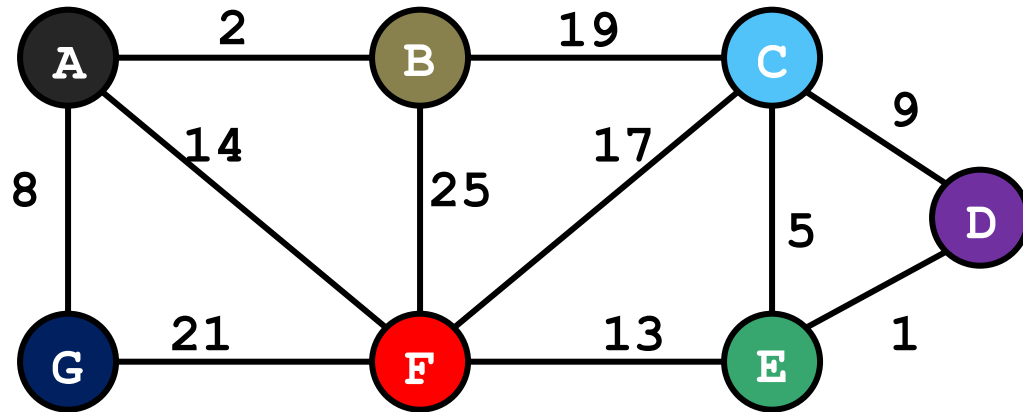


DISJOINT SET OPERATIONS

- makeSet() $O(1)$
 - Union(a,b) $O(\log n)$
 - findSet(a) $O(\log n)$
- (1,3)
 - (6,7)
 - (6,1)



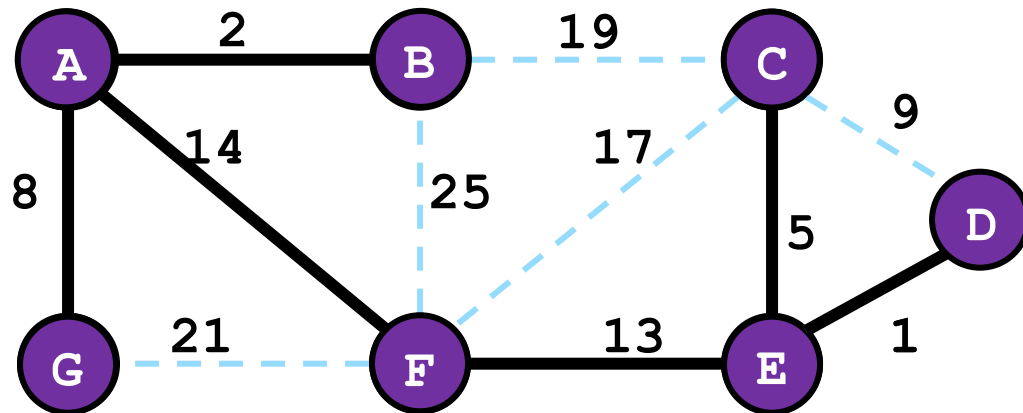
KRUSKAL's ALGORITHM (SIMULATION)



- Sort the edges in ascending order
- Create a set T for the MST with no edges

E — 1 — D
 A — 2 — B
 C — 5 — E
 A — 8 — G
 C — 9 — D
 E — 13 — F
 A — 14 — F
 C — 17 — F
 B — 19 — C
 G — 21 — F
 B — 25 — F

KRUSKAL's ALGORITHM (SIMULATION)



- Sort the edges in ascending order
- Create a set T for the MST with no edges
- Connect n-1 edges with no cycle
- Total cost: **43**

E	1	D	✓
A	2	B	✓
C	5	E	✓
A	8	G	✓
C	9	D	✗
E	13	F	✓
A	14	F	✓
C	17	F	
B	19	C	
G	21	F	
B	25	F	

DONE !



MakeSet

- ❑ Let there are n elements denoted from 1 to n
- ❑ Consider $n = 8$

1 2 3 4 5 6 7 8

- ❑ Create n disjoint sets for the n number of elements

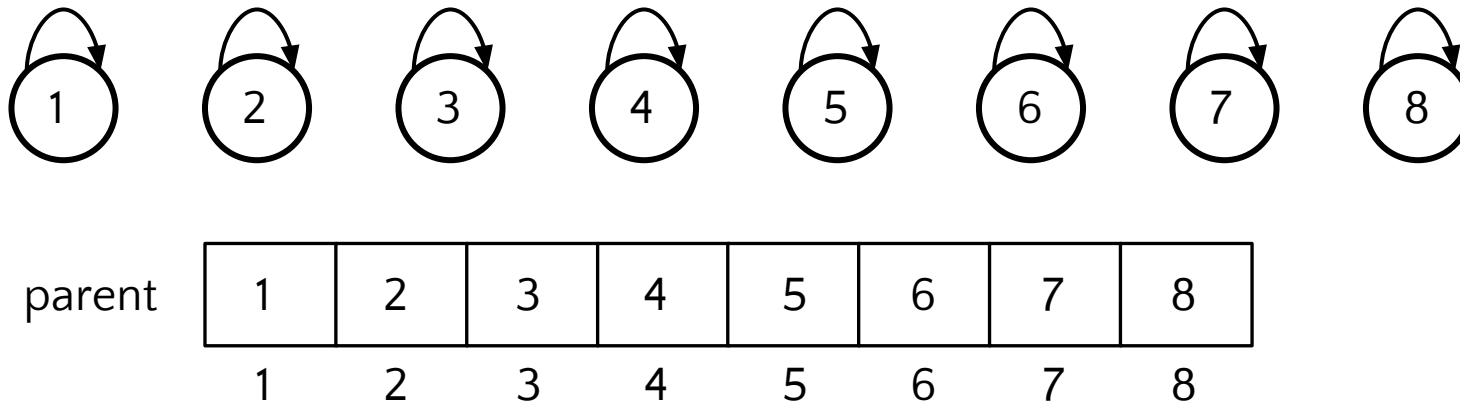
{1} {2} {3} {4} {5} {6} {7} {8}

- ❑ The only element is the representative of that set

- Representative({1}) = 1
- Representative({2}) = 2
- Representative({3}) = 3

MakeSet

- Let there are n elements denoted from 1 to n
- All the 8 elements belong to 8 disjoint sets
- $\text{makeSet}(i)$ sets the parent of i as i
- for($i=1$; $i \leq n$; $i++$) $\text{parent}[i] = i$;
- Consider $n = 8$
- Each set has an unique representative





Union

- ❑ Merges two sets and creates a single disjoint set

$\{1\}$ $\{2\}$ $\{3\}$ $\{4\}$ $\{5\}$ $\{6\}$ $\{7\}$ $\{8\}$

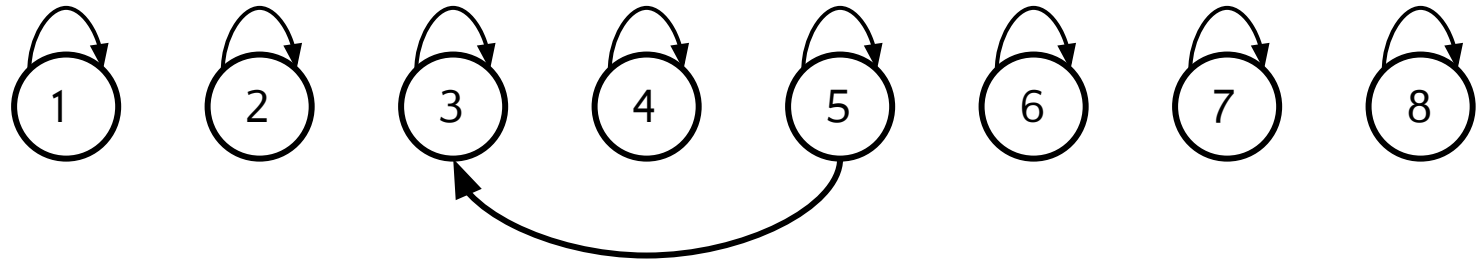
- ❑ Union(1,5) $\{1,5\}$ $\{2\}$ $\{3\}$ $\{4\}$ $\{6\}$ $\{7\}$ $\{8\}$

- ❑ Union(2,8) $\{1,5\}$ $\{2,8\}$ $\{3\}$ $\{4\}$ $\{6\}$ $\{7\}$

- ❑ Union(5,8) $\{1,5,2,8\}$ $\{3\}$ $\{4\}$ $\{6\}$ $\{7\}$

Union

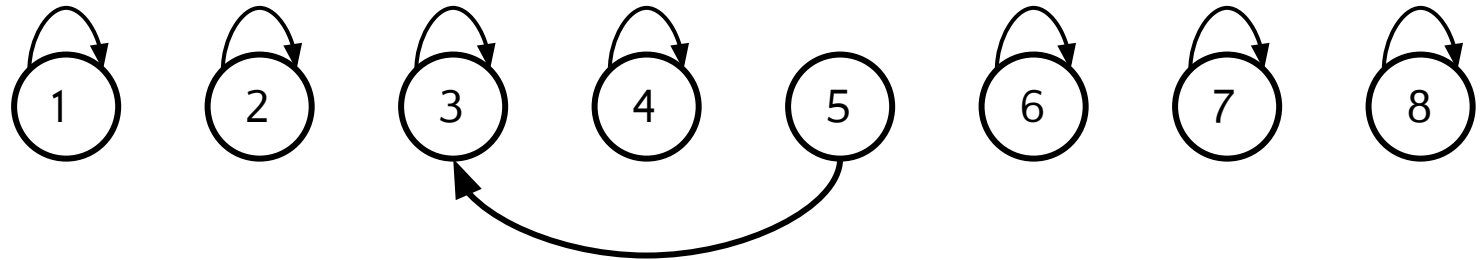
- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$



parent	1	2	3	4	5	6	7	8
	1	2	3	4	5	6	7	8

Union

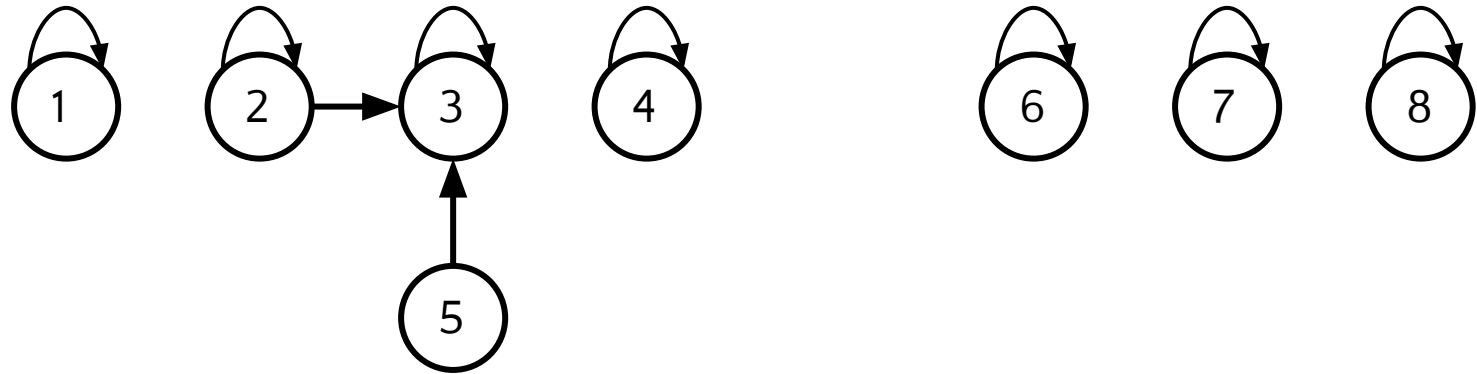
- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$



parent	1	2	3	4	3	6	7	8
	1	2	3	4	5	6	7	8

Union

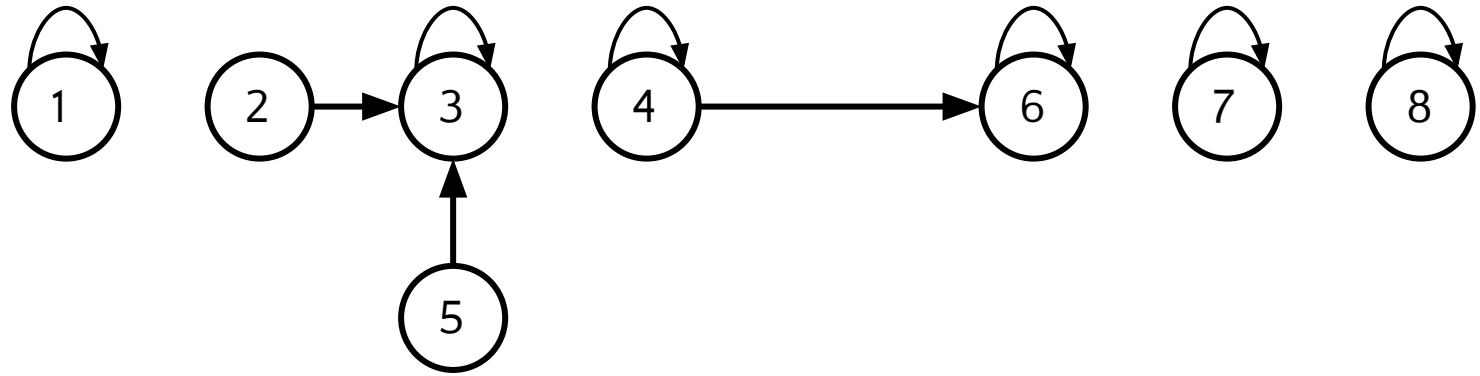
- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$
- $\text{union}(3,2)$



parent	1	2	3	4	3	6	7	8
	1	2	3	4	5	6	7	8

Union

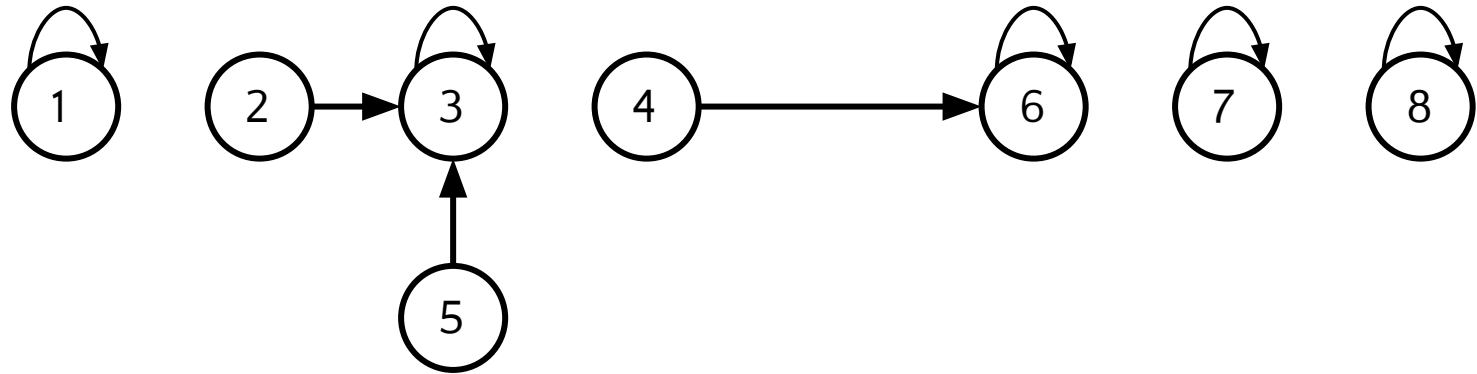
- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$



parent	1	3	3	4	3	6	7	8
	1	2	3	4	5	6	7	8

Union

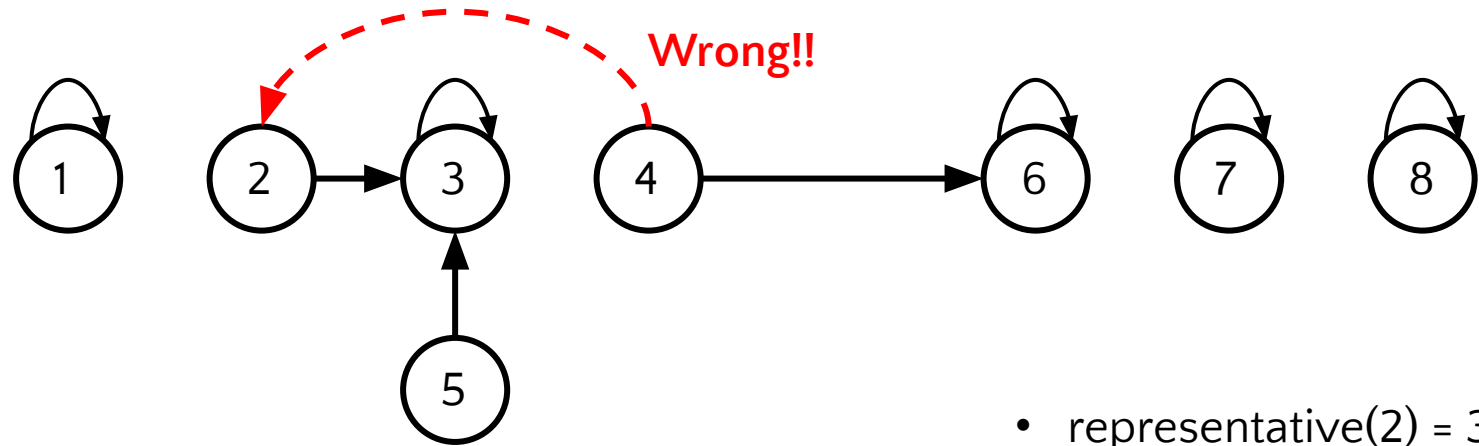
- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$



parent	1	3	3	6	3	6	7	8
	1	2	3	4	5	6	7	8

Union

- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$
- $\text{union}(2,4)$



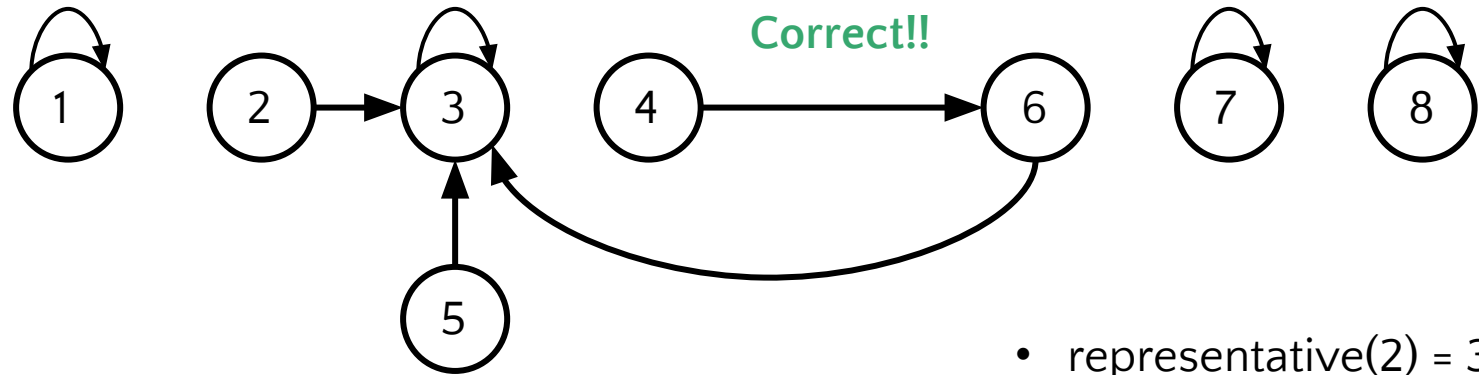
- $\text{representative}(2) = 3$
- $\text{representative}(4) = 6$

parent	1	3	3	6	3	6	7	8
	1	2	3	4	5	6	7	8

Union

- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**

- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$
- $\text{union}(2,4)$

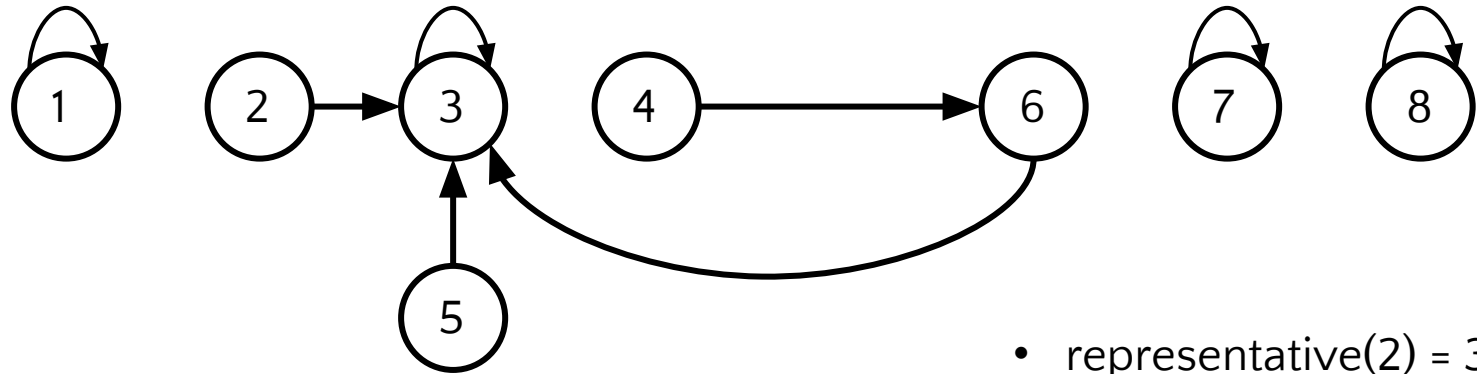


- $\text{representative}(2) = 3$
- $\text{representative}(4) = 6$

parent	1	3	3	6	3	6	7	8
	1	2	3	4	5	6	7	8

Union

- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$
- $\text{union}(2,4)$

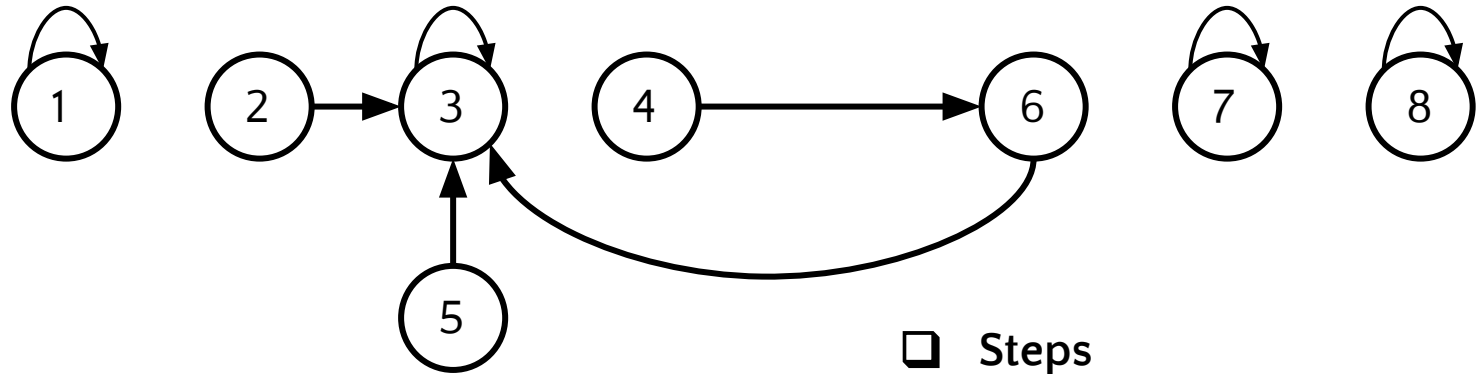


- $\text{representative}(2) = 3$
- $\text{representative}(4) = 6$

parent	1	3	3	6	3	3	7	8
	1	2	3	4	5	6	7	8

Union

- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$
- $\text{union}(2,4)$



Steps

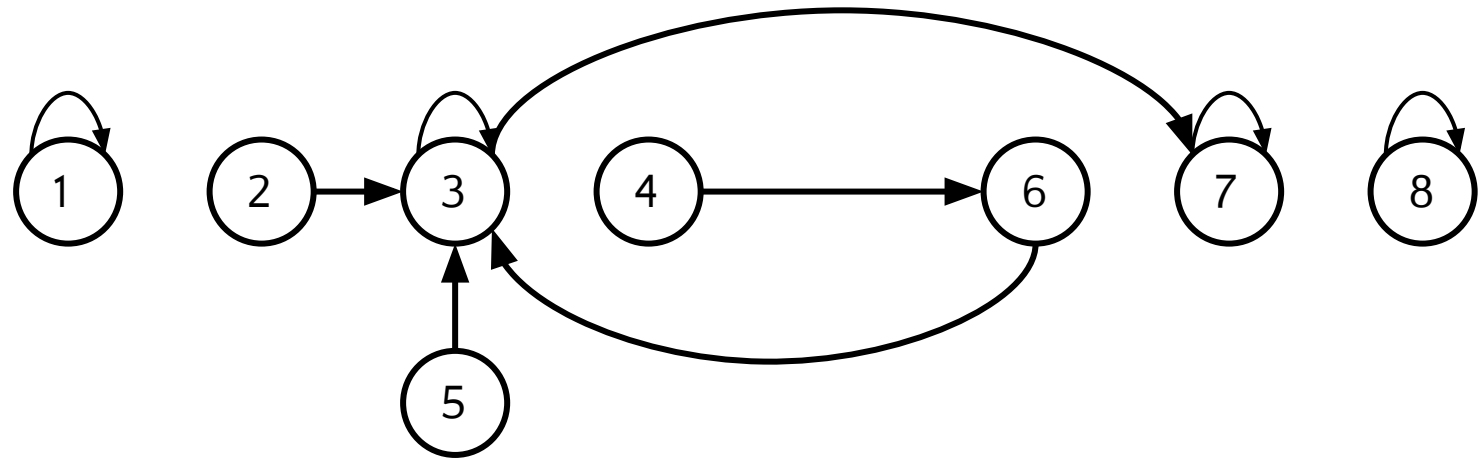
- $R_a = \text{findRepresentative}(a)$
- $R_b = \text{findRepresentative}(b)$
- $\text{parent}[R_b] = R_a$

parent	1	3	3	6	3	3	7	8
	1	2	3	4	5	6	7	8

Union

- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**

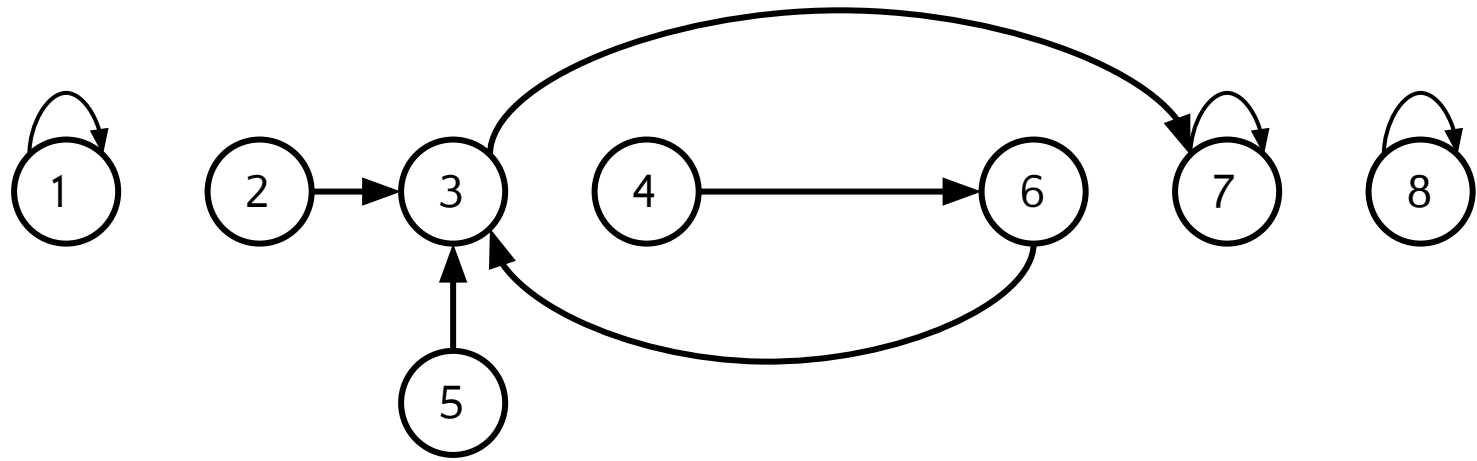
- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$
- $\text{union}(2,4)$
- $\text{union}(7,4)$



parent	1	3	3	6	3	3	7	8
	1	2	3	4	5	6	7	8

Union

- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$
- $\text{union}(2,4)$
- $\text{union}(7,4)$



parent	1	3	7	6	3	3	7	8
	1	2	3	4	5	6	7	8

Union

- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**

- $\text{union}(3,5)$

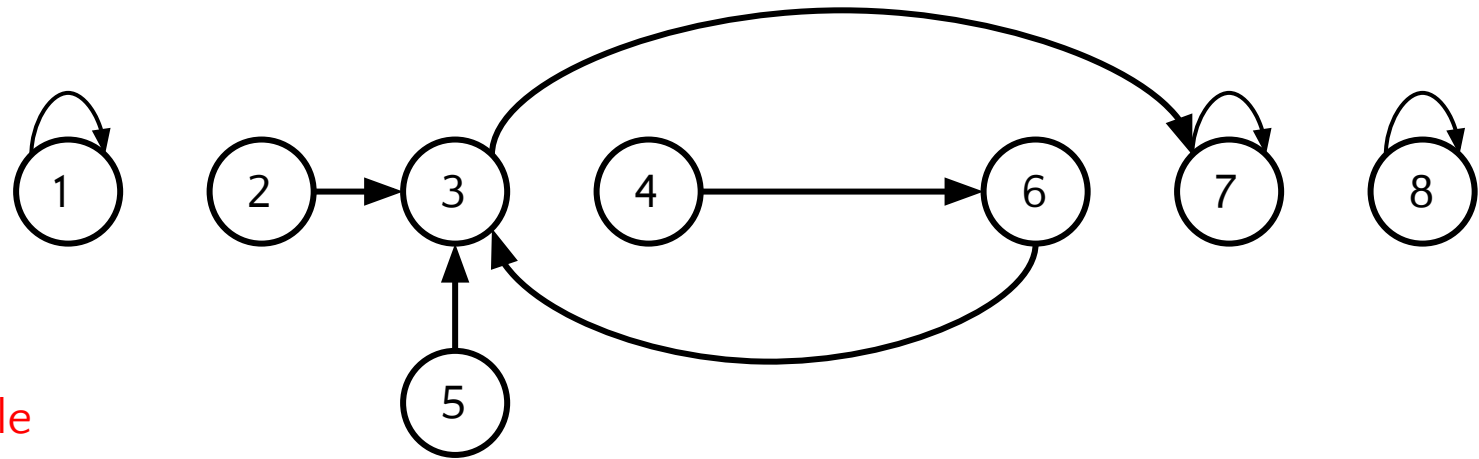
- $\text{union}(3,2)$

- $\text{union}(6,4)$

- $\text{union}(2,4)$

- $\text{union}(7,4)$

- $\text{union}(3,6)$ **X Not possible**

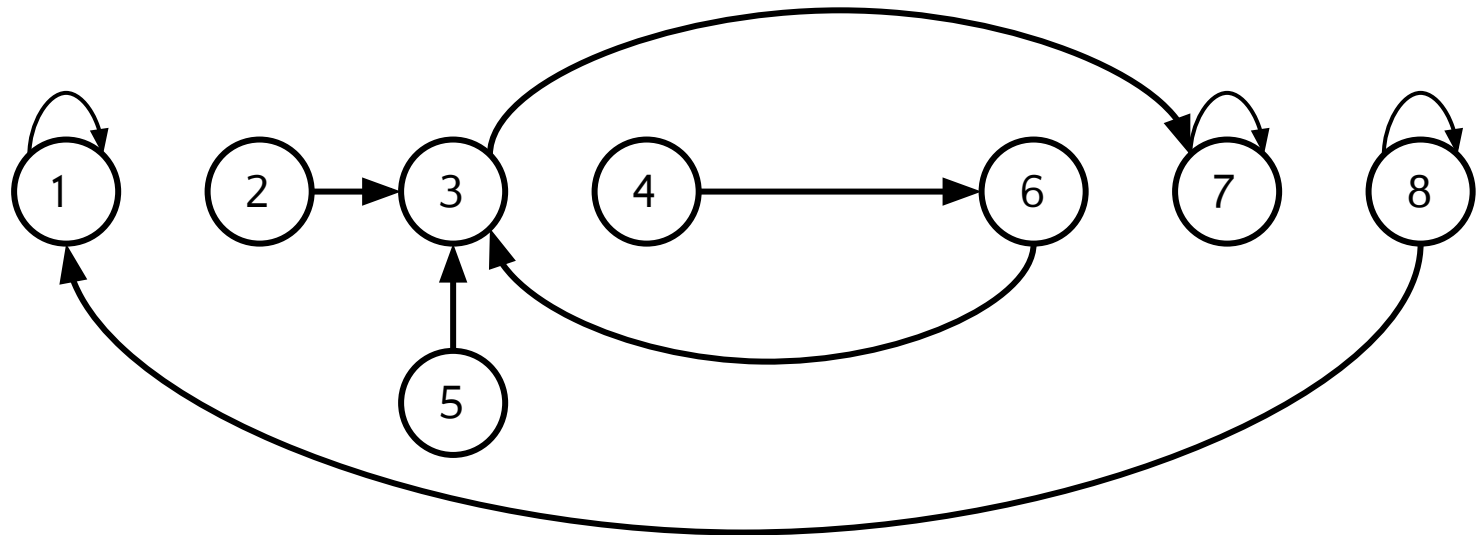


parent	1	3	7	6	3	3	7	8
	1	2	3	4	5	6	7	8

Union

- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**

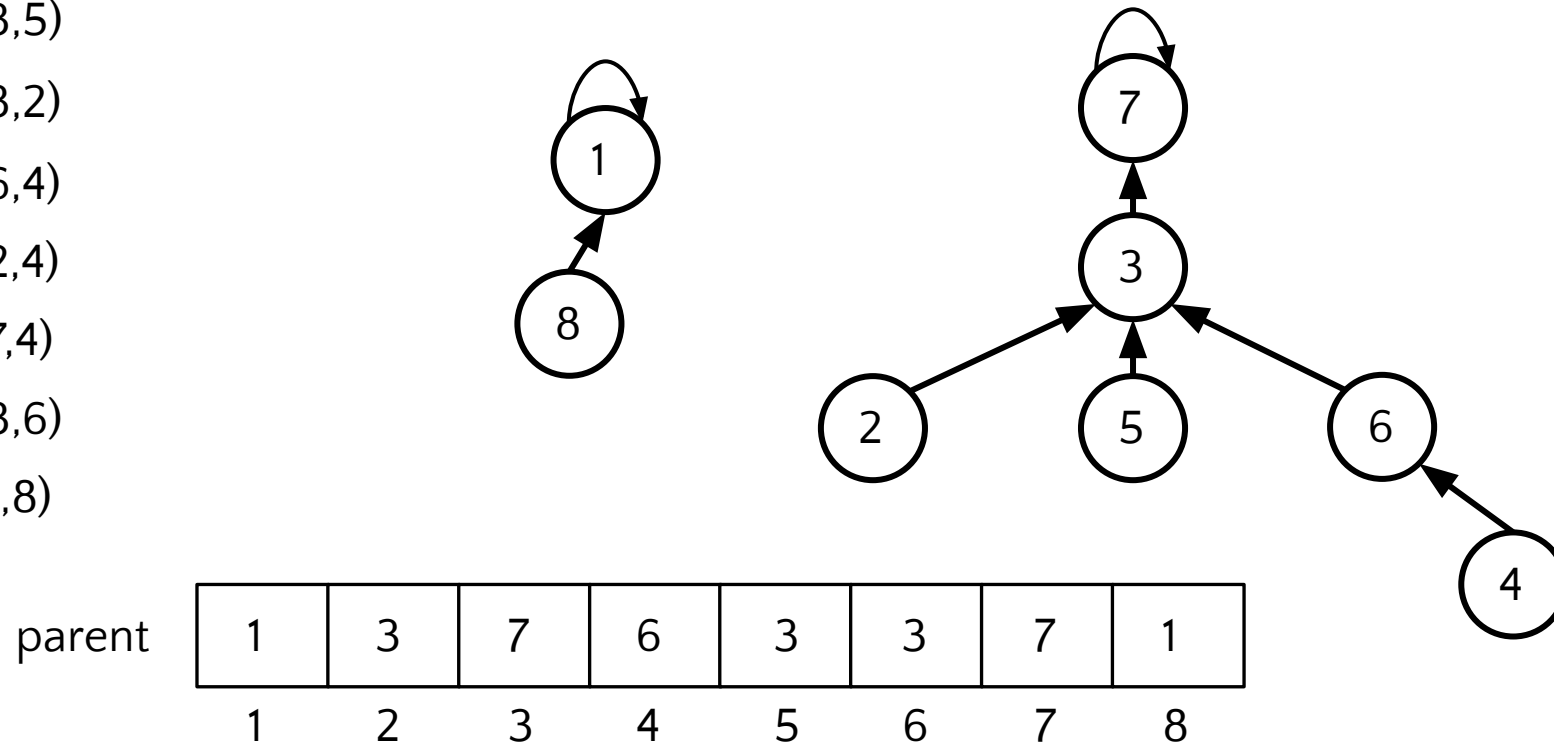
- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$
- $\text{union}(2,4)$
- $\text{union}(7,4)$
- $\text{union}(3,6)$
- $\text{union}(1,8)$



parent	1	3	7	6	3	3	7	8
	1	2	3	4	5	6	7	8

Union

- $\text{union}(a,b)$ brings b into the set of a if a and b belong different sets **otherwise union is not possible**
- $\text{union}(3,5)$
- $\text{union}(3,2)$
- $\text{union}(6,4)$
- $\text{union}(2,4)$
- $\text{union}(7,4)$
- $\text{union}(3,6)$
- $\text{union}(1,8)$





Union By Rank And Path Compression Heuristic

❑ Union By Rank

- Link by size
- Link by height

❑ Path Compression

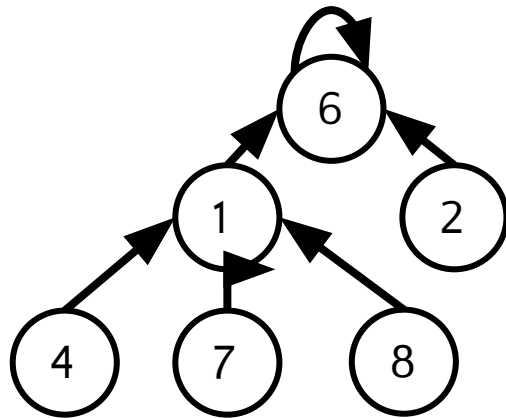
- Used in findSet function of a node x
- It not only finds the representative of x
- Also makes each node in the path directly point to the root

EXERCISE

- Illustrate the figure of the forests from the following *parent* array where *parent*[*v*] = *u* denotes that *u* is the parent of *v*

parent

6	6	3	1	3	6	1	1
1	2	3	4	5	6	7	8

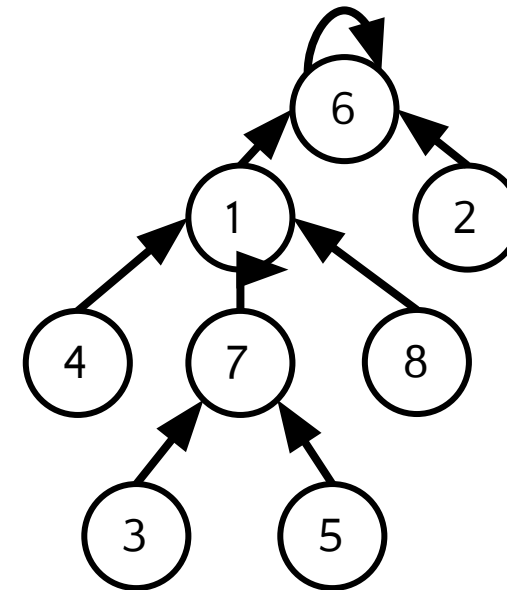


- Write down the sets in the disjoint set

- {1, 2, 4, 6, 7, 8}
- {3, 5}

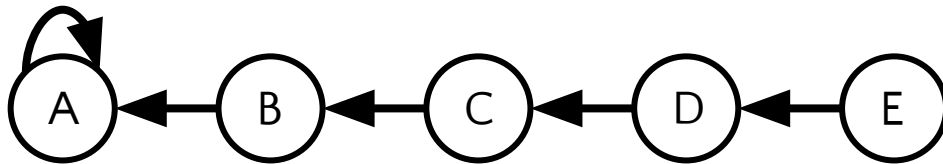
EXERCISE

- Consider the disjoint set at right
- What can be the time complexity for finding the representative of an element?
□ Ans: $O(h)$
 - What can be the maximum value of h ?
□ Ans: $n-1$



EXERCISE

- Consider a disjoint set of $n = 5$ elements denoted from 'A' to 'E'
- Do the union operations in such a way that height of the tree becomes $n-1$
 - $\text{union}(x, y)$ brings y into the set of x if x and y belong different sets **otherwise union is not possible**



- $\text{union}('D, 'E')$
- $\text{union}('C, 'D')$ Or $\text{union}('C, 'E')$
- $\text{union}('B, 'C')$ Or $\text{union}('B, 'D')$ Or $\text{union}('B, 'E')$
- $\text{union}('A, 'B')$ Or $\text{union}('A, 'C')$ Or $\text{union}('A, 'D')$ Or $\text{union}('A, 'E')$

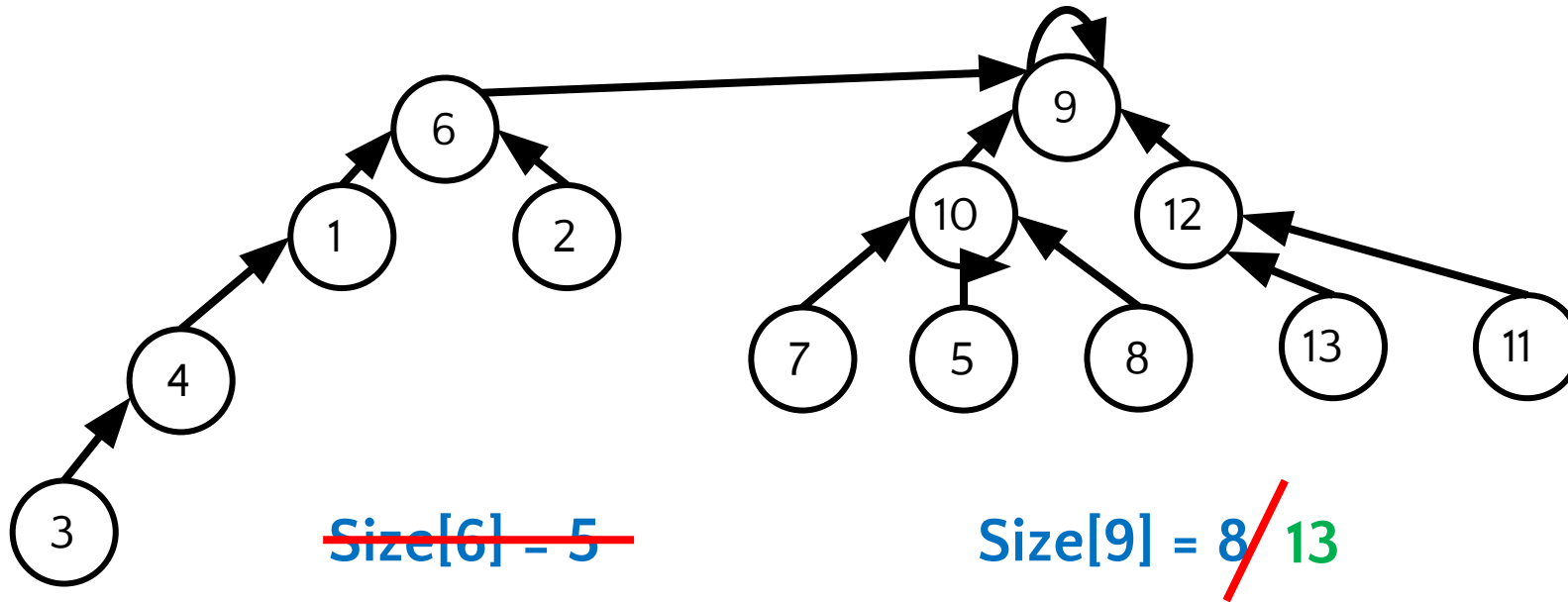


Link By Size

- ❑ If x and y belongs to two different sets then $\text{union}(x, y)$ does the following
 - Brings x into the set of y if $\text{set-size}(x) < \text{set-size}(y)$
 - Brings y into the set of x if **otherwise**

Link By Size

❑ What is the result of union(10, 4)?





Link By Size

```
union(x, y)
```

```
    fx := findRepresentative(x)
```

```
    fy := findRepresentative(y)
```

```
    if fx = fy then
```

```
        return
```

```
    if size[x] < size[y] then
```

```
        parent[x] = y
```

```
        size[y] = size[x] + size[y]
```

```
    else
```

```
        parent[y] = x
```

```
        size[x] = size[x] + size[y]
```

❑ Initial size?

- 1

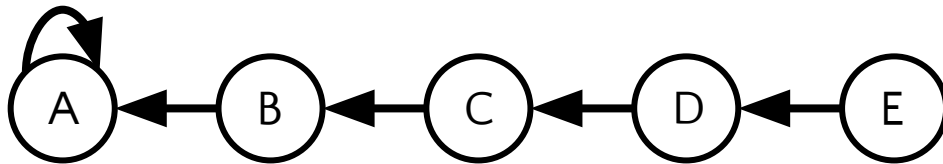
```
makeSet(i)
```

```
    parent[i] := i
```

```
    size[i] := 1
```

EXERCISE

- ❑ Consider a disjoint set of $n = 5$ elements denoted from 'A' to 'E'
 - Do the union operations in such a way that height of the tree becomes $n-1$
 - $\text{union}(x,y)$ uses link by size



- **Not possible!**

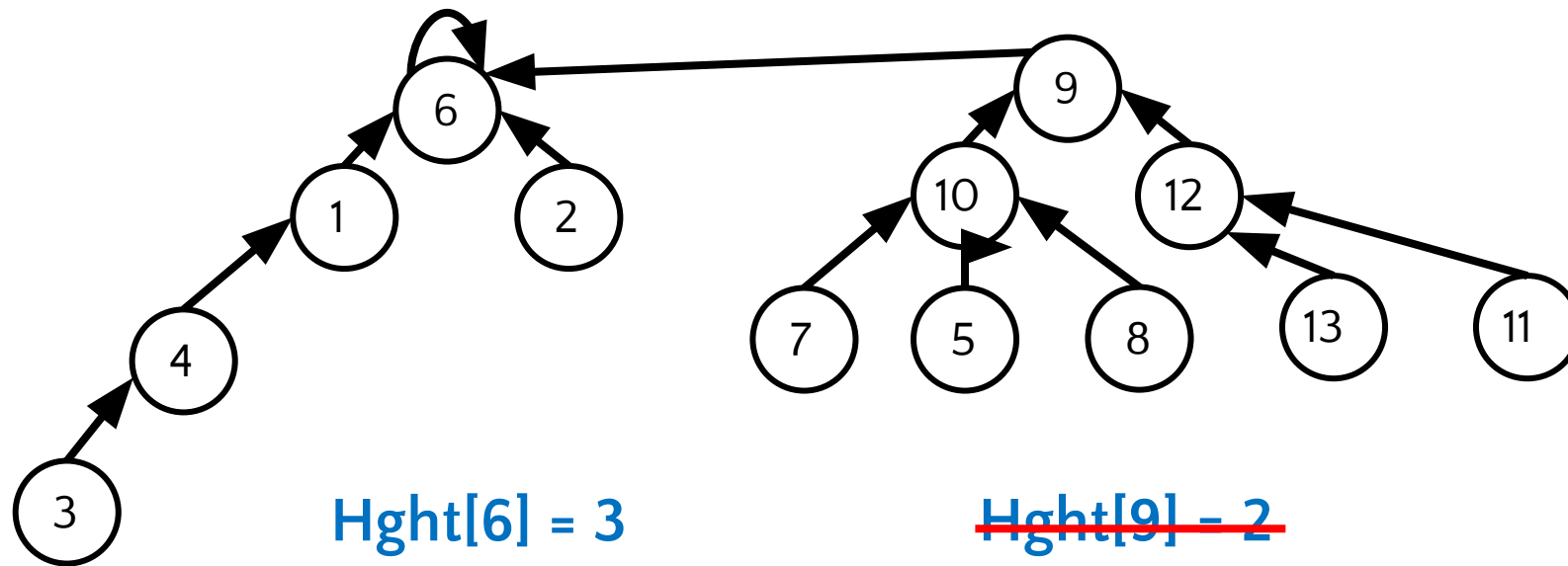


Link By Height

- ❑ If x and y belongs to two different sets then $\text{union}(x, y)$ does the following
 - Brings x into the set of y if $\text{tree-height}(x) < \text{tree-height}(y)$
 - Brings y into the set of x if **otherwise**

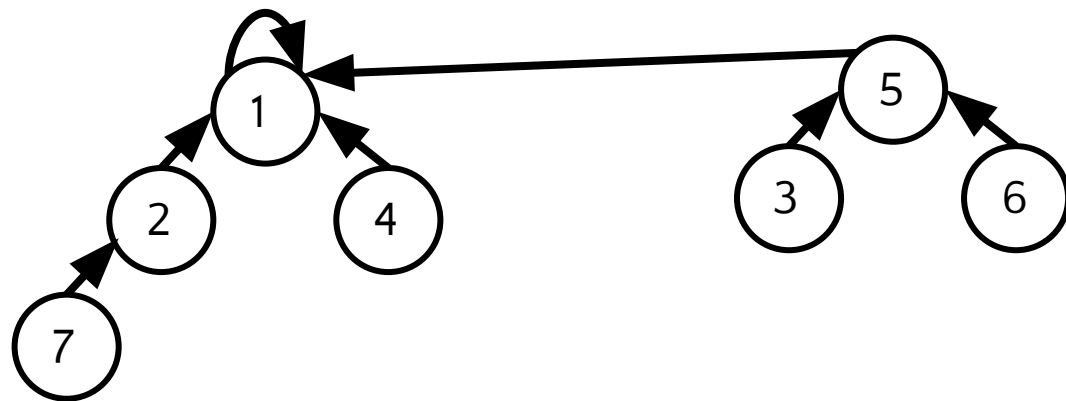
Link By Height

❑ What is the result of union(10, 4)?



Link By Height

❑ What is the result of union(2, 5)?

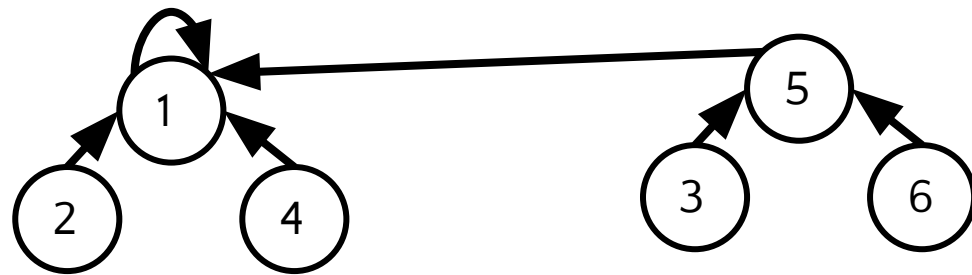


Hght[1] = 2

~~Hght[5] = 1~~

Link By Height

❑ What is the result of union(2, 5)?



$$\text{Hght}[1] = 1 / 2$$

$$\text{Hght}[5] - 1$$



Link By Height

```
union(x, y)
    fx := findRepresentative(x)
    fy := findRepresentative(y)
    if fx = fy then
        return
    if height[x] = height[y] then
        parent[x] = y
        height[y]++
    else if height[x] < height[y] then
        parent[x] = y
    else
        parent[y] = x
```

❑ Initial height?

- 0

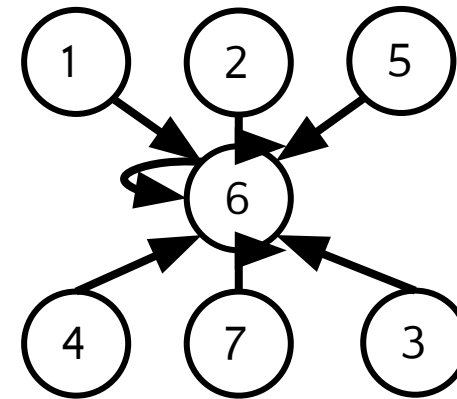
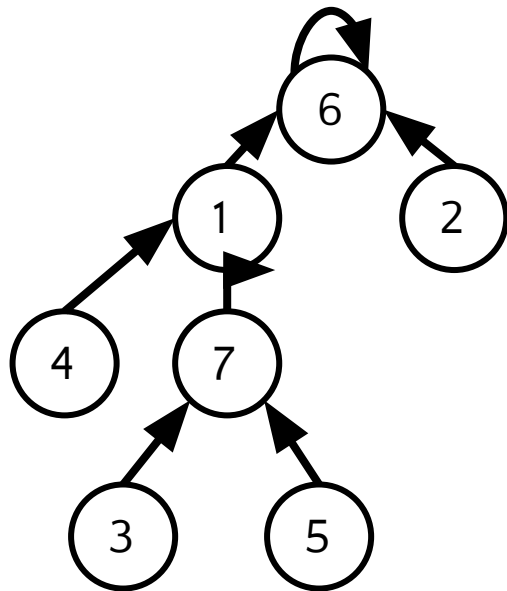
makeSet(i)

parent[i] := i

height[i] := 0

Exercise

- ❑ Which of the design is more preferable for optimizing the process of finding representative?



This one!

- Height is less
- Connectivity with root is high



Same Set Condition

- a and b belongs to same set if $\text{representative}(a) = \text{representative}(b)$

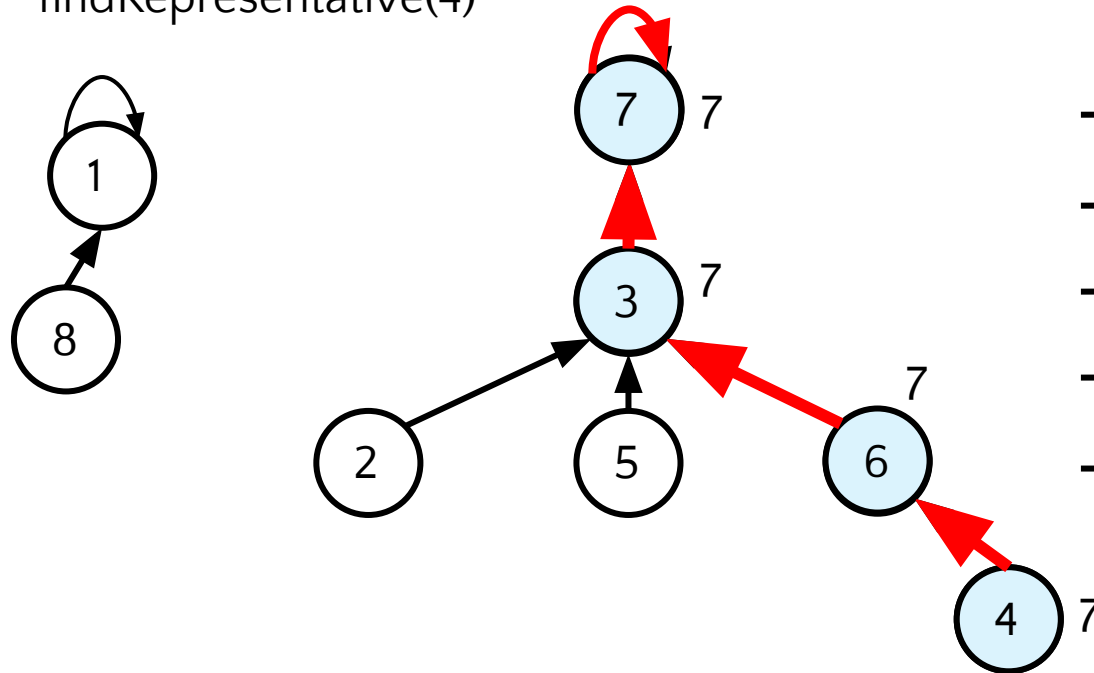


Representative Identification

- If for an element r we find $\text{par}[r] = r$ then r is definitely a representative of a disjoint set

FindRepresentative(x)

- findRepresentative(4)



parent	1	3	7	6	3	3	7	1
	1	2	3	4	5	6	7	8

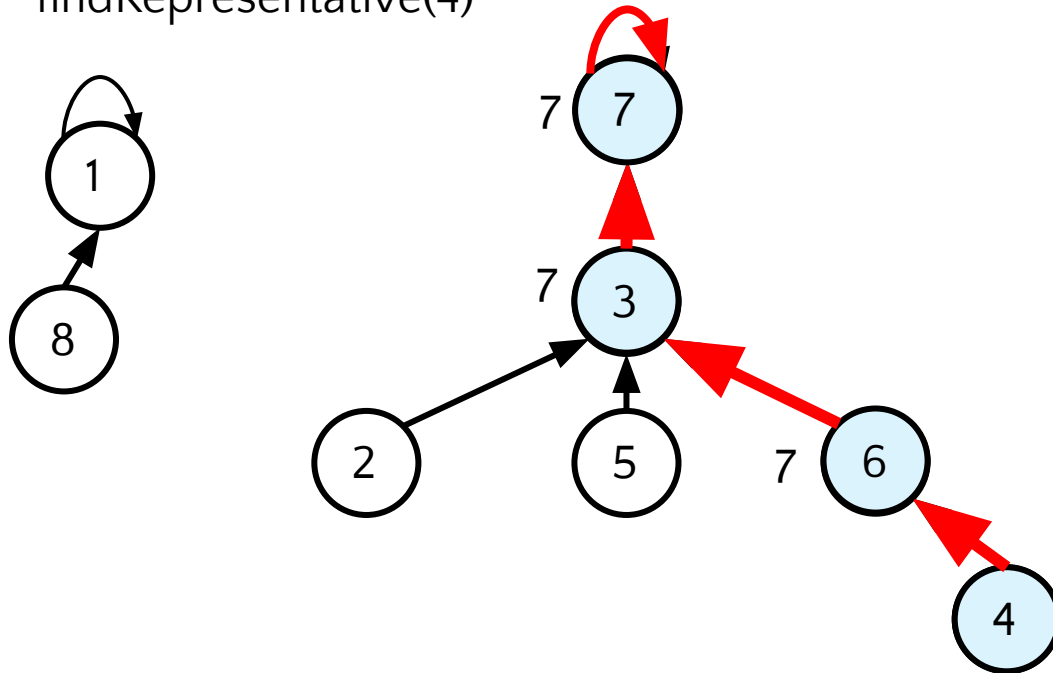
Recursion	x	parent[x]	Return
fR(4)	4	6	7
fR(6)	6	3	7
fR(3)	3	7	7
fR(7)	7	7	7

```

int fR(int x){
    if(x==parent[x]) return x;
    int r = fR(parent[x]);
    return r;
}
    
```

FindRepresentative with Path Compression

- findRepresentative(4)

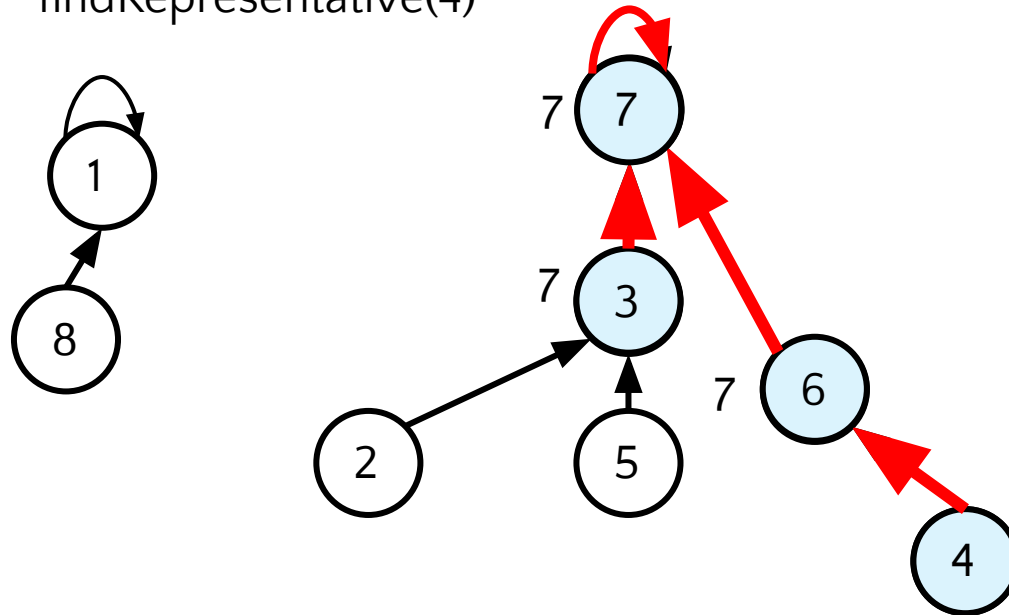


Recursion	x	parent[x]	Return
fR(4)	4	6	
fR(6)	6	3	7
fR(3)	3	7	7
fR(7)	7	7	7

parent	1	3	7	6	3	3	7	1
	1	2	3	4	5	6	7	8

FindRepresentative with Path Compression

- findRepresentative(4)

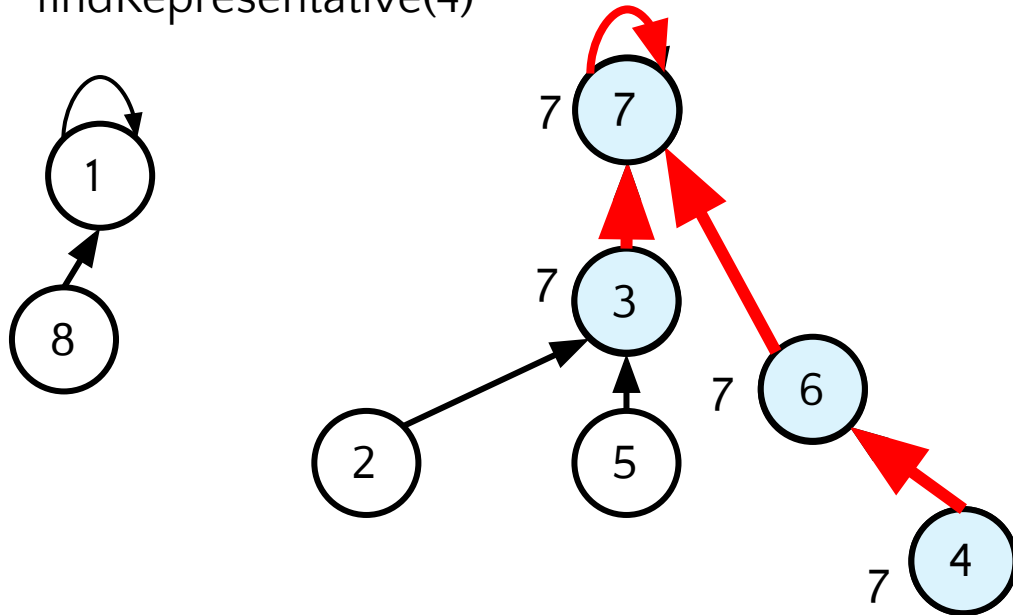


Recursion	x	parent[x]	Return
fR(4)	4	6	
fR(6)	6	7	7
fR(3)	3	7	7
fR(7)	7	7	7

parent	1	3	7	6	3	7	7	1
	1	2	3	4	5	6	7	8

FindRepresentative with Path Compression

- findRepresentative(4)

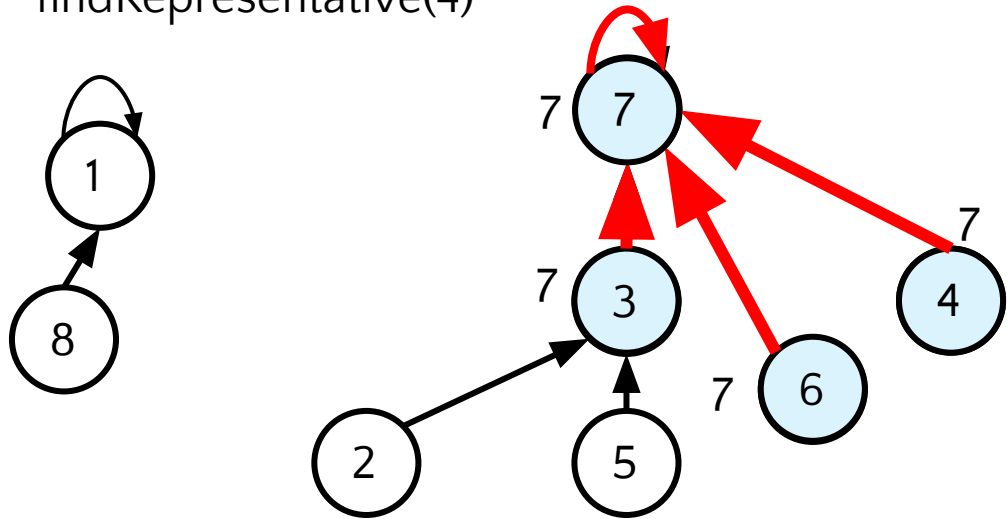


Recursion	x	parent[x]	Return
fR(4)	4	6	7
fR(6)	6	7	7
fR(3)	3	7	7
fR(7)	7	7	7

parent	1	3	7	6	3	7	7	1
	1	2	3	4	5	6	7	8

FindRepresentative with Path Compression

- findRepresentative(4)



Recursion	x	parent[x]	Return
fR(4)	4	7	7
fR(6)	6	7	7
fR(3)	3	7	7
fR(7)	7	7	7

parent	1	3	7	7	3	7	7	1
	1	2	3	4	5	6	7	8

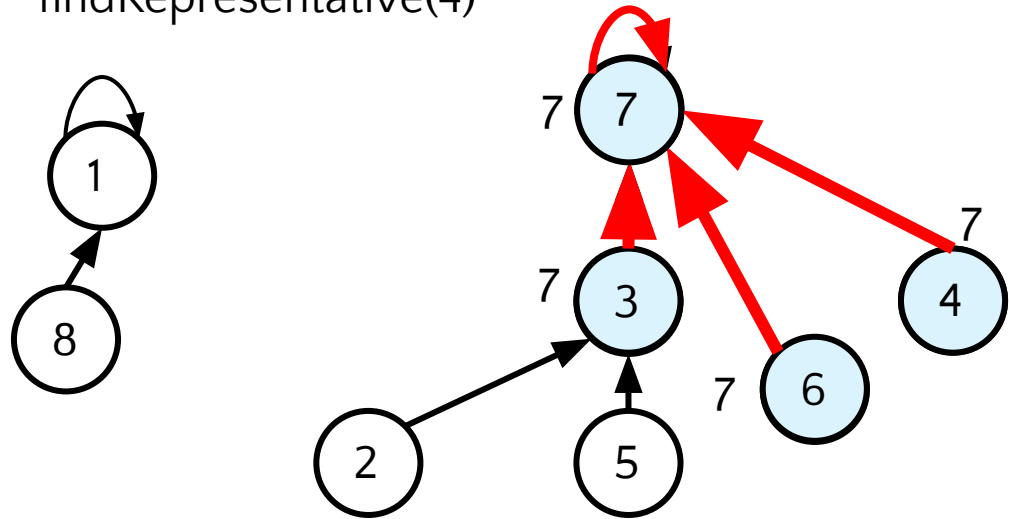
```

int fR(int x){
    if(x==parent[x]) return x;
    int r = fR(parent[x]);
    par[x] = r;
    return r;
}
    
```



FindRepresentative with Path Compression

- findRepresentative(4)



Recursion	x	parent[x]	Return
fR(4)	4	7	7
fR(6)	6	7	7
fR(3)	3	7	7
fR(7)	7	7	7

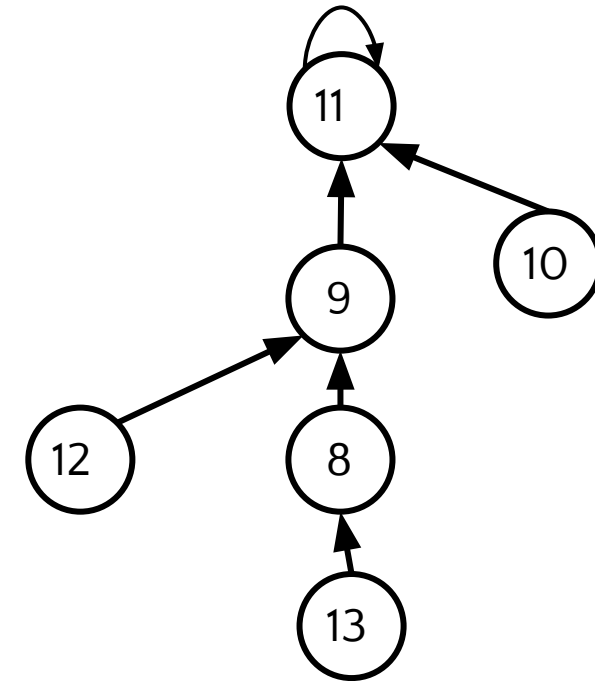
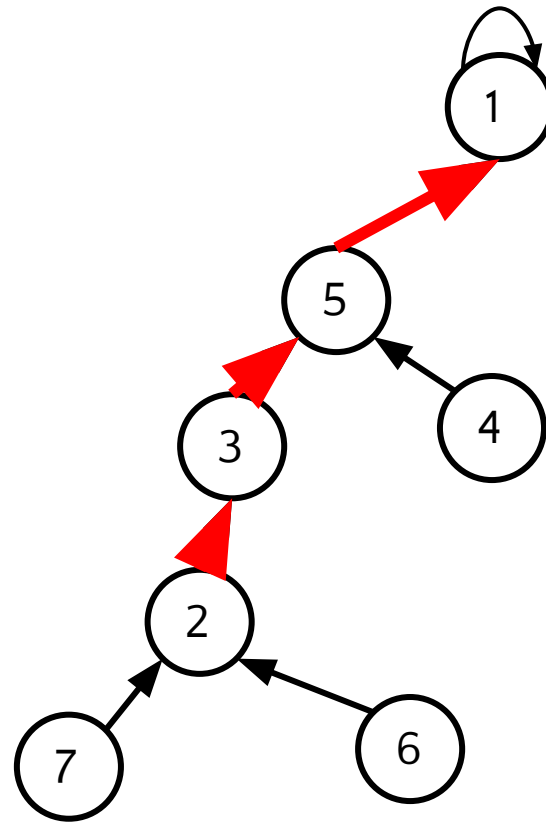
Now what is the complexity to find the representative of 4?

Ans: $O(1)$

parent	1	3	7	7	3	7	7	1
	1	2	3	4	5	6	7	8

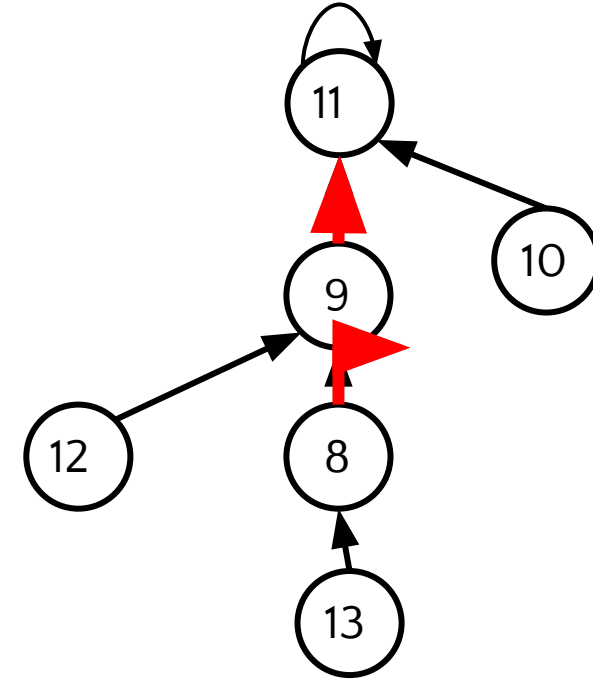
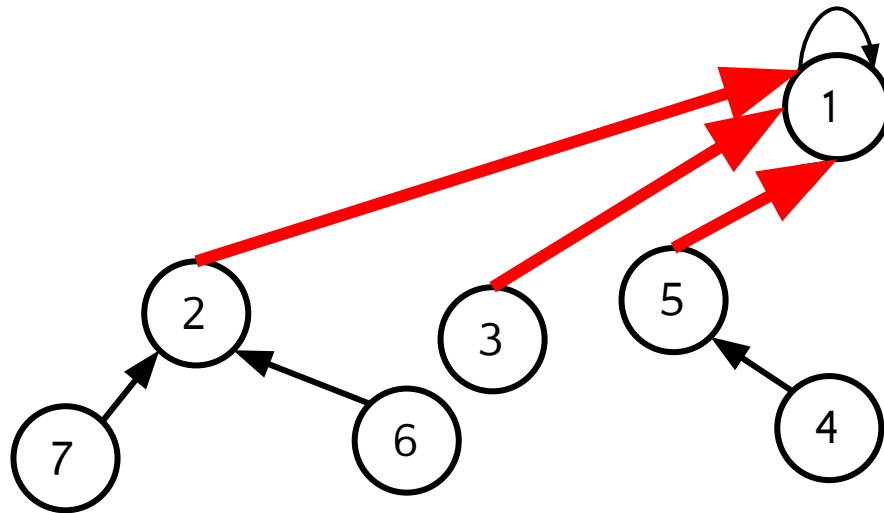
Union with Path Compression

- Union(2,8)
 - `int r1 = fR(2);`



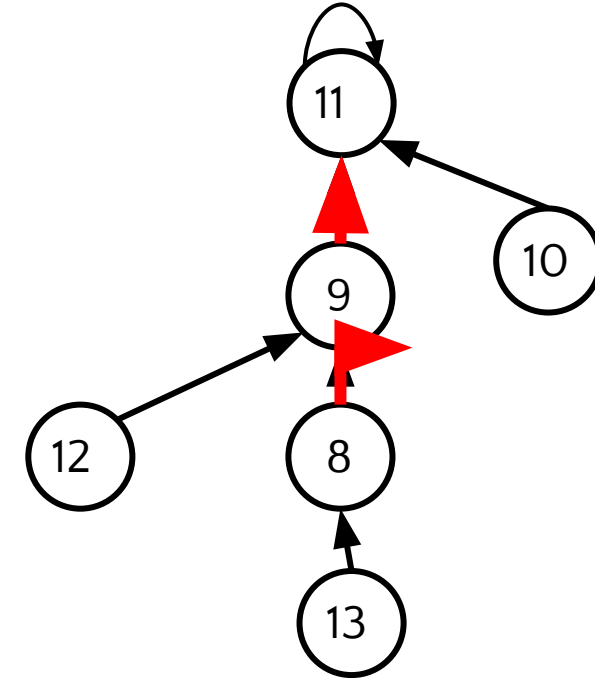
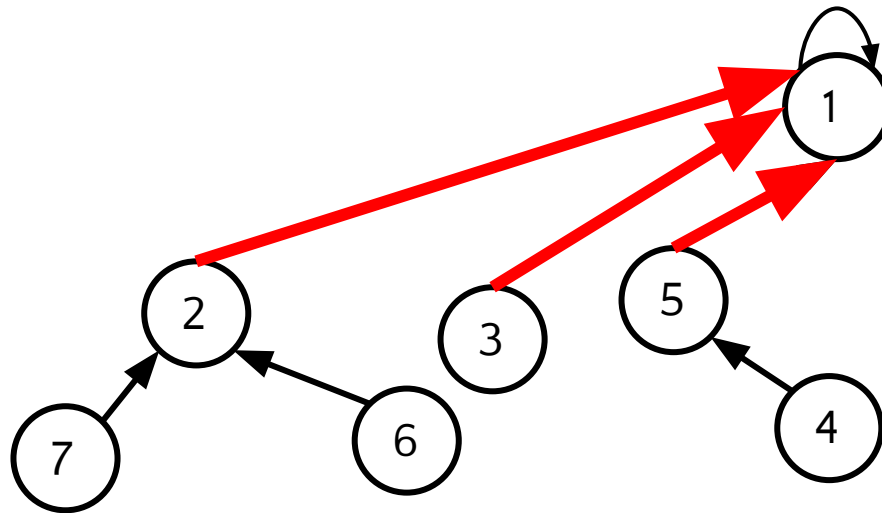
Union with Path Compression

- Union(2,8)
 - `int r1 = fR(2);` `[r1 = 1]`



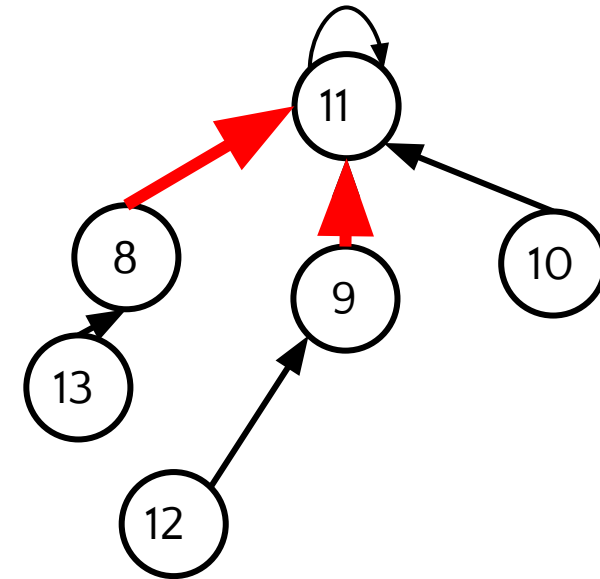
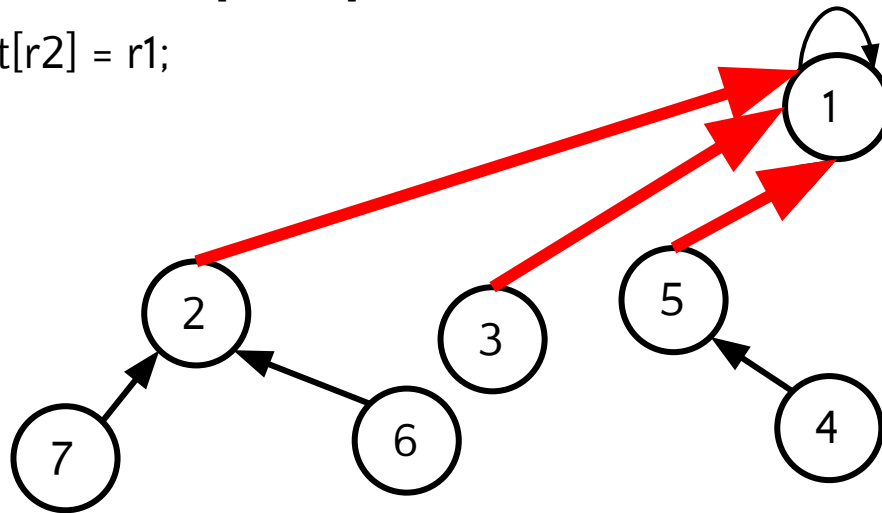
Union with Path Compression

- Union(2,8)
 - `int r1 = fR(2);` [`r1 = 1`]
 - `int r2 = fR(8);`



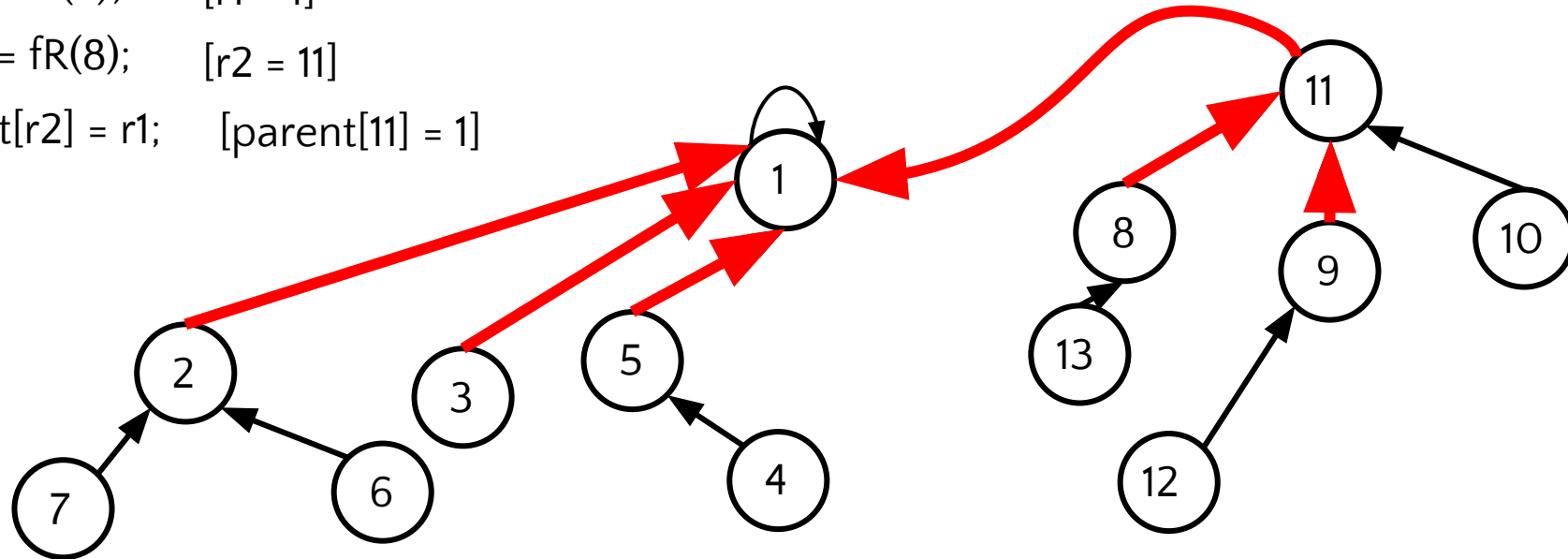
Union with Path Compression

- Union(2,8)
 - `int r1 = fR(2);` `[r1 = 1]`
 - `int r2 = fR(8);` `[r2 = 11]`
 - `parent[r2] = r1;`



Union with Path Compression

- Union(2,8)
 - `int r1 = fR(2);` [`r1 = 1`]
 - `int r2 = fR(8);` [`r2 = 11`]
 - `parent[r2] = r1;` [`parent[11] = 1`]





THANK YOU!

ANY QUESTION?